Energy and AI 14 (2023) 100276

Contents lists available at ScienceDirect

Energy and AI

journal homepage: www.elsevier.com/locate/egyai

Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents

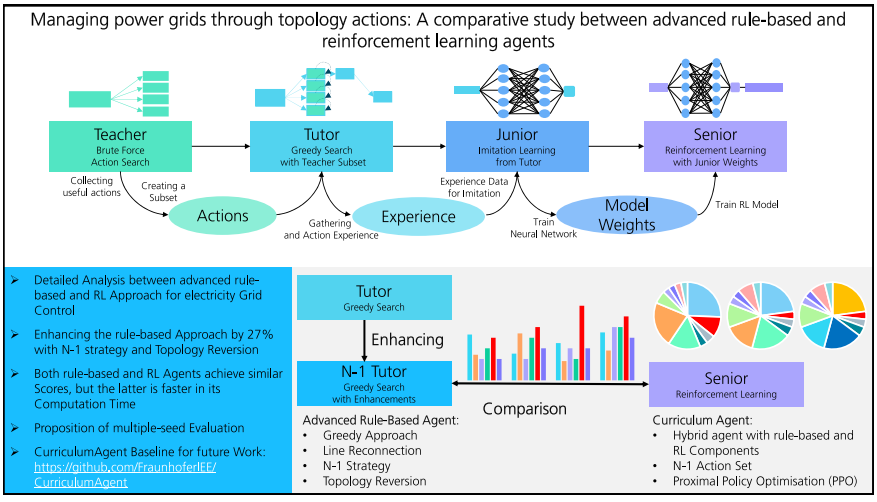
Malte Lehna [a,b,*], Jan Viebahn [c], Antoine Marot [d], Sven Tomforde [e], Christoph Scholz [a,b]

[a] Fraunhofer Institute for Energy Economics and Energy System Technology (IEE), Germany
[b] Kassel University: Intelligent Embedded Systems, Germany
[c] TenneT TSO BV, Netherlands
[d] AI Lab, Reseau de Transport d'Electricite (RTE), France
[e] Kiel University: Intelligent Systems, Germany

HIGHLIGHTS

- Comparison between rule-based and reinforcement learning agents for grid control.
- The performance of the rule-based agent could be improved through a N-1 strategy.
- Reversion to the original topology proofed to be advantageous for the grid stability.
- Similar agent performance but faster computation time by reinforcement learning agent.
- The CurriculumAgent package is flexible for all Grid2Op environments.

GRAPHICAL ABSTRACT



ARTICLE INFO

Dataset link: https://github.com/FraunhoferIEE/CurriculumAgent

Keywords:
Deep reinforcement learning
Electricity grids
Learning to run a power network
Topology control
Proximal policy optimisation

ABSTRACT

The operation of electricity grids has become increasingly complex due to the current upheaval and the increase in renewable energy production. As a consequence, active grid management is reaching its limits with conventional approaches. In the context of the Learning to Run a Power Network (L2RPN) challenge, it has been shown that Reinforcement Learning (RL) is an efficient and reliable approach with considerable potential for automatic grid operation. In this article, we analyse the submitted agent from Binbinchen and provide novel strategies to improve the agent, both for the RL and the rule-based approach. The main improvement is a N-1 strategy, where we consider topology actions that keep the grid stable, even if one line is disconnected. More, we also propose a topology reversion to the original grid, which proved to be beneficial. The improvements are tested against reference approaches on the challenge test sets and are able to increase the performance of the rule-based agent by 27%. In direct comparison between rule-based and RL agent we find similar performance. However, the RL agent has a clear computational advantage. We also analyse the behaviour in an exemplary case in more detail to provide additional insights. Here, we observe that through the N-1 strategy, the actions of both the rule-based and the RL agent become more diversified.

* Corresponding author at: Fraunhofer Institute for Energy Economics and Energy System Technology (IEE), Germany.
E-mail address: malte.lehna@iee.fraunhofer.de (M. Lehna).

https://doi.org/10.1016/j.egyai.2023.100276
Received 2 April 2023; Received in revised form 31 May 2023; Accepted 2 June 2023
Available online 5 June 2023

1. Introduction

1.1. Overview

With the growth of renewable energies in the electricity mix and their volatile behaviour, the complexity of operating an electricity grid is constantly increasing for Transmission System Operators (TSO) [1]. As a result, it is not only necessary to plan production capacities and predict the demand correctly, but also consider new options to manage grids in times of instabilities. Topological changes at substation level are an option that is gaining increasing attention as this is an existing cheap but underutilised flexibility. Their usage in controlling the stability of electricity grids has been discussed by several researchers over the last decades [2]. However, changes in topology also have a major drawback, namely that their optimisation requires a large amount of computational resources [3]. In the past, Optimal Power Flow (OPF) optimisation has often been used, but researchers have noted potential difficulties with the increase in renewable energy and smart grids [4] as well as criticised its inability to cope with large, non-linear combinatorial action spaces [5]. To solve this problem, the French TSO RTE proposed with the Learning to Run a Power Network (L2RPN) a Reinforcement Learning (RL) challenge that aims to join forces between the grid experts and the Machine Learning (ML) community.[1] In the challenge, [6] provided a RL environment with the Grid2Op package [7], which allowed research on real data, while allowing the training of different algorithms on electricity grids.[2] Grid2Op has become one of the leading frameworks for RL grid control in scenario-based simulations. We therefore conduct our research in this environment in order to provide comparable results.

1.2. Research contribution

While the first contributions to the automation of grid control have been made [8], it is not yet clear how large the impact of the RL solution will be. Is the RL approach solely responsible, or can heuristic and rule-based approaches achieve a similar solution if their behaviour is sophisticated enough?

In our work, we contribute to this discussion by providing a systematic analysis between a rule-based agent and a RL agent. For this, we have chosen the framework of Binbinchen [9] from the 2020 L2RPN robustness challenge, as it includes both a rule-based and a RL agent. As part of our work, we extend the rule-based approach with two significant improvements. First, we propose an N-1 strategy to ensure more robust topology actions of the rule base agent. This N-1 strategy favours topology actions that ensure a stable grid, even if one line of the grid is disconnected. Second, we encourage the agent to revert to the original topology when the state of the grid is relatively stable. We analysed the effect of these improvements in an experiment, where we ran the scenarios with 30 different seeds to ensure an significant evaluation with less randomness. Our experiments show that with our improvement we were able to increase the performance of the rule-based agent by 27% in comparison to the original Binbinchen agent. Furthermore, we are able to show that our advanced rule-based approach is able to achieve similar performance to the RL agent. However, when analysing the computational cost, the RL agent is still advantageous. In order to use the agents on all Grid2Op environments, we reworked the original code and published it as an open-source package on Github.[3] Overall, our contributions can be summarised as follows:

1. We analysed the solution of [9] and transferred its contents from a rapid prototyping state to a Python package. With various algorithmic changes, we ensure the usability for other grids without hard-coded implications based solely on the robustness challenge. This allows the community to include methods from the CurriculumAgent and compare their agents against our baseline.
2. We introduce two important improvements (N-1 strategy and topology reversion) to the rule-based agent that dramatically increase its performance by up to 27% in comparison to the original agent.
3. We apply the above modifications to the RL agent and benchmark its performance with the rule-based method.
4. We propose a test framework with multiple seeds to ensure significant comparable results.

The remainder of this article is structured as follows. In Section 2, we provide the related research and afterwards outline the structure of the Grid2Op environment in Section 3. In Section 4, we then introduce the *Teacher–Tutor–Junior–Senior* framework, followed by our novel improvements in Section 5. In Section 6, we present the experimental setup of this article and afterwards, in Section 7, we discuss the results. Lastly, we provide a conclusion in Section 8.

2. Related work

In recent years, there has been an increasing interest in the usage of Reinforcement Learning (RL) for the control of power systems. One early paper was published by [10] in 2004, where the authors proposed to use the RL approach for both offline and online applications in the context of power systems. However, the use of RL was still constrained due to the computational limitations of that time. Everything changed with the breakthrough of Deep Reinforcement Learning (DRL), first introduced by [11,12]. Their work set off a wave of research, with ground breaking results in various fields [13–15]. The idea behind DRL is that the policies and/or value functions of the algorithms are not computed directly, but instead are approximated through a deep neural network. This allows the learning of advanced strategies, even for complex problems.[4]

Following these breakthroughs, researchers began addressing grid control problems with RL, especially driven by the introduction of the L2RPN challenge [5,8,16,17]. The first successful RL solution was presented by participants in the L2RPN challenge 2019 [18], who used the Dueling Deep Q Network (DDQN) agent as their RL approach and were able to win the competition. The authors in [18] also introduced for a first time an imitation learning strategy and combined it with an guided exploration approach. In the following L2RPN WCCI 2020, other participants implemented a Semi Markov Afterstate Actor Critic (SMAAC) algorithm [19], which focused on the final topology instead of the switching of individual actions at the substation level to learn the different topology actions. By applying a Graph Neural Network (GNN) to the observation space, [19] further transformed the original grid into an afterstate representation. With their approach, [19] outperformed all other candidates and became the first winner of the challenge. Following the first challenge, [5,8] increased the difficulty for RL agents in the L2RPN NeurIPS 2020 challenge, by introducing a robustness track

[1] L2RPN challenges: https://l2rpn.chalearn.org/ (last access 20/03/2023).

[2] Grid2Op: https://github.com/rte-france/Grid2Op (last access 20/03/2023).

[3] CurriculumAgent: https://github.com/FraunhoferIEE/CurriculumAgent (last access 20/03/2023).

[4] Note that for the sake of simplicity, we henceforth refer to DRL as RL in this paper.

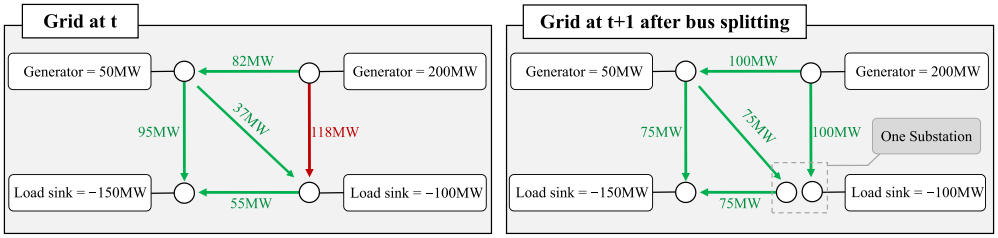


Fig. 1. Visualisation of an exemplary topology action, adapted from the paper of [6]. The original grid shows an overload of the right line (in red) at time step t, due to an high demand from both load sinks. By executing a topology action, i.e., splitting the load flow of the substation into two separate notes, the bottom-right substation can divert the power and the grid returns to a more stable state without an overflow. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

with an adversarial agent and an adaptability track with an increasing share of renewable energy injections [5]. Interestingly, neither track could be solved by the previous agents, instead new solutions were required. One submission was published in [20], where the authors used an evolutionary RL approach in order to win both challenge tracks. They introduced a planing algorithm to actively search through the available actions provided by the policy. Afterwards, the planing algorithm was optimised through an evolution strategy. Within the black box optimisation method, Gaussian white noise was added to ensure an adequate exploration of the policies. Apart from to topology actions, the authors also included line reconnecting actions and some hand picked redispatch actions [20]. The second best performance of the robustness track was the agent by Binbinchen [9], already mentioned in Section 1.2. They proposed a framework, where a RL agent learned though imitating a rule-based agent, ensuring a better handling of the large action space. From a methodological perspective, their agent was based on the Proximal Policy Optimisation (PPO) algorithm by [21]. Due to their performance and their straightforward framework, the approach of [9] is the foundation of our work and we describe their framework in more detail in Section 4.1.

One notable approach that was not part of the challenge, but instead was published recently, was the paper [22]. The researchers combined a RL algorithm with an heuristic approach, sharing several similarities to the agent of [9] and our work. With their combination, they were able to achieve better results than any previous agents on the legacy data set of the 2020 L2RPN robustness challenge, while at the same time reducing the overall run-time of their agent. Recently, another agent was proposed in [23], based on the AlphaZero [24]. Similar to AlphaZero, the researchers of [23] used the Monte Carlo Tree Search to find appropriate actions in the grid. However, some specifications were changed to the use-case of the grid management. These changes were an early stopping criteria to tackle the expensive computational costs and a heuristic model. The latter was used instead of a neural network to describe the agent's value function, further reducing the training time [23]. With their approach, they were not only able to win the L2RPN WCCI challenge of 2022, but also achieved with only topology actions the same results as agents with redispatching actions. This is indicates the importance of topology actions for future research.

Finally, given that we are also interested in rule-based approaches, we would like to emphasise the expert agent of [25]. In their work, the researchers build an agent that searched a priori for remedies by identifying local power paths to manage the electricity grid. In their agent, they include expert knowledge to rank the topology actions and chose the correct action in case the grid is unstable. Considering that the agent is also a good starting point for our rule-based approach, we include the agent as baseline in our work.

3. The Grid2Op environment

The Grid2Op environment is a Python package created by RTE to provide a platform for the development of RL agents. As outlined in [6,16], the package enables training and the evaluation of agents in the RL Gym framework [26] on known IEEE synthetic power grids, e.g., the IEEE14 or IEEE118 grids.[5] To correctly depict an Markov Decision Process (MDP), the environment of Grid2Op consists of an observation space, an action space and provides multiple reward function. The action space is divided into three action types. The first type of action is the line action $a^{(line)} \in \mathcal{A}^{(line)}$ that connects and disconnects lines between two substations. The second type of action is a topological action $a^{(topo)}$ from the set of all possible topology actions $a^{(topo)} \in \mathcal{A}^{(topo)}$. The topology action changes the node configuration on a substation level. This is visualised in Fig. 1, where the node splitting is demonstrated on a simple example case, based on [6]. Note that the complexity increases with more lines connected to a substation [17]. As a third type of action, Grid2Op offers the possibility to change the power injection through redispatch actions with $a^{(redisp)} \in \mathcal{A}^{(redisp)}$, by adjusting the production of the generators in the grid. While the action spaces $\mathcal{A}^{(line)}$ and $\mathcal{A}^{(topo)}$ are each discrete action spaces, $\mathcal{A}^{(redisp)}$ is considered continuous. With respect to the observation space, Grid2Op provides various information on the grid that range from topology information and power flows to line capacity and time variables as well as the injection of the generators and demand of the consumers. While all the information is vital in the training of the agents, especially the capacities of the lines are used to measure the stability of the grid. In terms of Grid2Op, the capacity of a line is defined as the observed current flow of the line, divided through its thermal limit. We denote the capacity of a line l as $\rho_{l,t} \in \mathbb{R}^+$, with $l = 1, \dots, L$ from the set of all lines $l \in \mathcal{L}$. We further recorded for each time step t the maximum capacity across all lines as $\rho_{max,t} = \max_{l=1,\dots,L}(\rho_{l,t})$. The grid is considered unstable, if at least one line is above its capacity of 100%. In addition to the actual observation, Grid2Op offers the possibility to simulate the effect of an action on the grid. This simulation method is not fully accurate, as it relies on forecasted values. We denote this simulation of the line capacity as $\hat{\rho}_{l,t+1}$ and the maximum of the simulated value as $\hat{\rho}_{max,t+1} = \max_{l=1,\dots,L}(\hat{\rho}_{l,t+1})$.

In our work, the robustness track of the 2020 L2RPN robustness track was chosen as benchmark. Here, the grid consisted of one out of three regions from the IEEE118 grid, shown in Fig. 2. This corresponds to an observation space of size 1429 and an action space with 59 line actions (discrete actions), 66,918 topology actions (discrete actions) and 80 redispatch actions (continuous actions). The key challenge of the L2RPN robustness track, however, was an adversarial agent described in [27]. The adversarial agent disconnected lines in the grid quasi-randomly, simulating unforeseeable moments in the grid. In the challenge, the number of lines has been limited to a total of ten target lines. The robustness environment further includes specific constraints to ensure a realistic setting of the challenge [6]. One constrain was that there were several reasons for a power line to disconnect from a substation. One could be a power overflow, i.e., $\rho_{max,t} > 1.0$, which would lead to a disconnection after three consecutive timesteps. Another were external line outages, either planed in the case of maintenance, or

[5] Gym-framework: https://www.gymlibrary.dev/ (last access 20/03/2023).

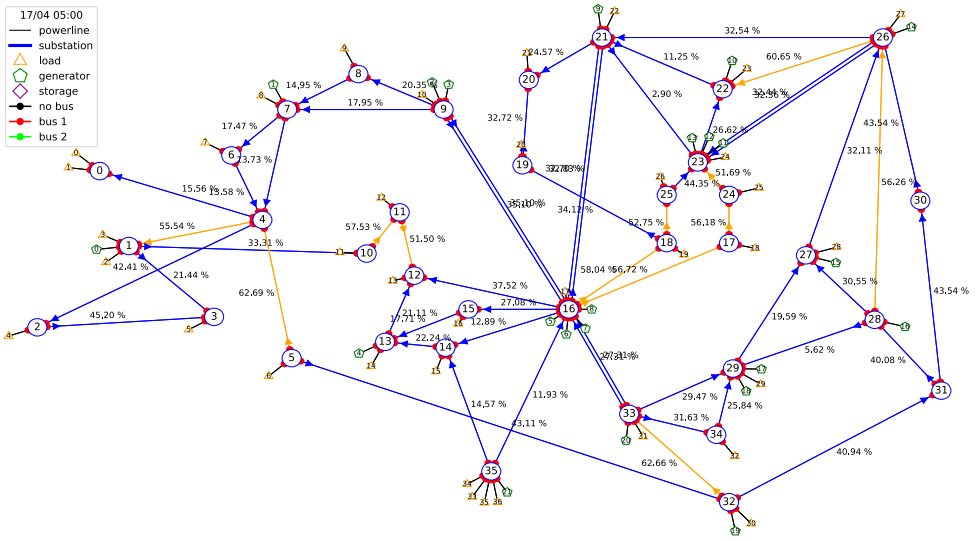


Fig. 2. The electricity grid of the robustness track, based on a subset of the IEEE118 grid. In the grid, a total of 35 substations exist that are interconnected with power lines. The grid has both generators and load sinks in different parts of the grid. As original state, all power lines are connected to bus 1 on the substations. Through topology actions, these can be changed to bus 2. The figure was created with the internal plot method of Grid2Op.

unplanned through adversarial attacks or other failures. Further, there were also constrains with regard to the action of the agent. Agents could not repeat an action on the same target (line, substation, redispatch) and therefore had to wait for a specific cooldown period. Again, as outlined in Section 2, there were differences between the cooldown periods of forceful line disconnection and active line disconnection by the agent, leading to imbalance in the game mechanics.[6] The agents had to adapt to the adversarial agent and these constrains, given that a missing line could trigger a cascading failure elsewhere in the grid [6]. In terms of the evaluation metric, we decided to use the same score as the L2RPN challenge, described in [8]. The score ranges from $[-100, 0, 80, 100]$, where -100 corresponds to an initial failure, 0 corresponds to the survival of the Do-Nothing Agent and 80 indicates a completion of all scenarios. The maximum score of 100 could be achieved, if the agents were further able to optimise the economical cost of their actions. Consequently, a higher score usually indicates a longer survival in the scenarios. For the testing, we were able to receive the test scenarios of the original challenge, thanks to the courtesy of the L2RPN organisers.

4. The Teacher-Tutor-Junior-Senior framework

4.1. Solution by Binbinchen

With the Grid2Op configuration outlined above, we analysed Binbinchen's solution [9]. They proposed a *Teacher–Tutor–Junior–Senior* framework in which a RL agent was trained through imitation learning of a rule-based agent. In the first step, the *Teacher* method used a brute force approach to gain experience and select topological actions by simulating all 66,918 actions. The most frequently used actions of the *Teacher* were then selected (208 by Binbinchen). With the reduced set of actions, the rule-based *Tutor* gathered experience consisting of observation–action pairs. The pairs were afterwards used as ground truth in a supervised learning context for imitation learning. The pairs were fed into the *Junior* model, which was a feed-forward network. In the last step, the RL approach, called *Senior*, was trained on an adapted Grid2Op environment. The *Senior* itself was started with the weights of the Junior model to accelerate faster training times and convergence.

We decided to analyse this approach in depth for the following reasons: First, in this paper we focus on unitary topology actions, i.e., a single topology action that changes the grid, rather than multiple consecutive actions. This is because we want the agent to identify importance based on its own strategy, rather than giving it a pre-selected order. According to [8], both the first place agent [20] and the third place agent [28] used more sequential approaches. Further, [20] combined their actions with redispatching actions. In this work, we only allow topology actions and line actions in order to reconnect disconnected lines, thus favouring the Binbinchen approach. As a second reason, we chose the framework because of its clear structure and comprehensibility. In the case of [20], interpretation is rather difficult, as the underlying method is based on a black-box optimisation approach. This is supported by the fact that Binbinchen's code is fully available, which allows for in-depth analysis and replicability. Finally, another participant reached second place with an adapted version of the Binbinchen agent in the 2021 L2RPN Trust challenge [29]. Therefore, the *Teacher–Tutor–Junior–Senior* framework is explained in detail in the following.

4.2. The Teacher

Although the robustness path of Grid2Op has a total of 66,918 topological actions, not all actions are feasible. Looking at Fig. 2, one can see that substation 16 is connected to several other substations, resulting in many possible combinations. In fact, more than 97.9% of all topology actions are topology configurations at substation 16. It is therefore clear that there is a need to reduce the number of actions. Thus, the *Teacher* model searches through all available actions in a brute force manner and selects the best candidate. More precisely, the *Teacher* iterates through different scenarios of the environment until the threshold of $\rho_{teacher} = 0.925$ is exceeded by $\rho_{max,t}$.[7] In that case,

[6] In a real life scenario the power lines require inspections before reconnection, thus the forcefull line disconnection had a cooldown of at least ten timesteps, while the line disconnection of the agent only had three. One timestep corresponds to five minutes in the simulation.

[7] Note that activation thresholds differ between the *Teacher*, *Tutor* and *Senior*. These ρ_{agent} are an important variable that can be used to fine-tune the response of the agents. In most cases, these thresholds have been chosen by Binbinchen in their submission of the challenge. We have verified these values in preliminary experiments and could confirm that these thresholds produce the best behaviour.

the *Teacher* searches through all actions and evaluates them with the pre-build simulation method of the Grid2Op environment. The action which results in the lowest $\hat{\rho}_{max,t+1}$ is selected, executed and saved (excluding do-nothing actions). After simulating a large number of scenarios, all the saved actions are evaluated. The actions are sorted by their frequency, i.e., the number of occurrences across the scenarios. Afterwards, one manually selects a subset of the most frequent actions. Next to the simple brute force approach, it is possible to adjust the searching algorithm to gather more advanced actions. Binbinchen created an adversarial *Teacher* that forcefully disconnected relevant lines, simulating more frequently the behaviour of the adversarial agent. This resulted in two different action sets of Binbinchen with 62 actions from the adversarial *Teacher* and 146 of the normal *Teacher*. In our work, we also use these 208 actions and build our enhancements on top.

4.3. The Tutor

With the subset of the *Teacher*, a rule-based agent is created to generate experience for the following stages of the framework. The respective *Tutor* is already a fully functioning rule-based agent that iterates over various scenarios of the Grid2Op environment and saves both the observations, as well as the selected actions as experience.[8] In order to achieve realistic behaviour, several rules are implemented for the *Tutor*. First, the agent does not interact with the grid, if $\rho_{max,t}$ is below the *Tutor* threshold of $\rho_{tutor} = 0.9$. Second, if the threshold is breached, the *Tutor* iterates over all 208 actions with a greedy approach. For each action, the *Tutor* checks whether the action is valid and then selects the action which results in the lowest simulated $\hat{\rho}_{max,t+1}$. Third, if a line is disconnected and there is no cooldown time remaining, the agent automatically reconnects the respective line. With this rule-based approach, Binbinchen were able to achieve a score of 44.69 on the online data set, which is already quite satisfactory, when comparing it to their RL approach with a score of 52.42. In our work, we build on top of the general *Tutor* structure and developed three additional enhancements that will be discussed in Section 5.

4.4. The Junior

The third component of the framework is the *Junior* agent, which is a feed-forward network that imitates the behaviour of the *Tutor*. The design of the *Junior* is fairly simple: The input layer of the network has the shape of the *Tutor* observation, while the output layer has the shape of all 208 topological actions. In the original agent from Binbinchen, the network consists of four layers à 1000 neurons (Relu-Activation) to process the data. In our work, we executed a hyperparameter search, which resulted in a different number of neurons, as seen in Appendix B.

With regard to the performance, the *Junior* is able to correctly predict the right *Tutor* action with around 37% accuracy, which is relative low. However, when considering the top 20 actions, the correct action is found with an accuracy of 92%. After the network is trained, the *Junior* is used to jump-start the *Senior* model.

4.5. The Senior

Finally, the RL *Senior* agent has a similar neural network architecture to the *Junior* model, i.e., the same layer and neuron structure as well as input and output shapes. The model is trained with the state-of-the-art RL algorithm PPO [21]. At the beginning of the training, the model is first initialised with the weights of the *Junior* model. Because the agent is only required to act in critical situations, the training of the *Senior* had to be adjusted. No action is taken if $\rho_{max,t}$ is below the threshold of $\rho_{senior} = 0.9$, with the exception of line reconnection if applicable. Is the threshold breached, the current state is passed to the *Senior* and the model has to chose a valid action. In the training, the *Senior* collects the basic Grid2Op reward, however accumulated over all previous steps where the do-nothing action was selected.

After convergence, the RL model is combined with the heuristic strategies (do-nothing and line reconnection actions) to create the final agent. Again, an action of the RL model is only required, if the threshold is breached. In this case, the model returns the probabilities of the RL policies and sorts the list of actions. The list is then reviewed one by one until a suitable candidate is found. Overall, one could therefore consider the final agent as a hybrid between the rule-based approach and the RL model.

5. Methodological improvements of the existing agent

As outlined in the research contributions in Section 1.2, we propose multiple enhancements of the *Teacher–Tutor–Junior–Senior* framework of Binbinchen [9], with a primary focus on the *Tutor* agent. Based on their code, we revised and enhanced the framework and created the CurriculumAgent package.[9] The enhancements were threefold and are presented in the following.

5.1. N-1 strategy improvements

As a first improvement, we propose to prioritise topology actions that especially reduce the overall $\rho_{max,t}$ in the event of a line failure, e.g., through a lightning stroke or adversarial attacks on lines. This corresponds to the well-known principle of N-1 security, already established among TSO operators. We search for these topological actions by creating a special N-1 *Teacher* as well as an N-1 *Tutor* that prioritises the execution of the N-1 actions.

The underlying pseudo-code of the N-1 algorithm is given in Algorithm 1 and can be described as followed: We start the N-1 search for a subset of lines of size M and all topological actions $a_i^{(topo)} \in \mathcal{A}^{(topo)}$ with $i = 1, \dots, N$. For each available topological action $a_i^{(topo)}$, we iterate over each line l in the subset $S \subseteq \mathcal{L}$, with $l = 1, \dots, M$ and create an action $a_l^{(line)} \in \mathcal{A}^{(line)}$ that disconnects the line. By combining both actions $a_{i,l} = a_i^{(topo)} \wedge a_l^{(line)}$, we can then simulate the observation of the next step. With the simulated line capacities $\hat{\rho}_{j,t+1}^{(i,l)} \in \hat{\mathcal{P}}$ for all lines $j = 1, \dots, L$, we have the combined expected effect of the topology action $a_i^{(topo)}$ and the disconnection of line l. Consequently, we can then calculate the maximum value of the grid $\hat{\rho}_{max,t+1}^{(i,l)}$.[10]

Afterwards, we record the maximum value across all lines $\hat{\rho}_{max}^{(i,max)}$, which are the worst possible line overloads for the topological action $a_i^{(topo)}$, when one of the lines is disconnected. Consequently, by sorting all $\hat{\rho}_{max}^{(i,max)}$ in ascending order, it is possible to get the best N-1 action that is available in the respective observation, i.e. $a_i^{\star} = \text{argmin}(\hat{\rho}_{max}^{(i,max)})$. Note, however, that the N-1 calculation is computationally intensive. If expert knowledge is available it can be beneficial to only select a subset of the lines.

5.2. Topology reversion improvement

As a second improvement, the topology reversion proved to be another essential component in developing a more advanced greedy agent. This improvement is based on the idea that the electricity grid is most stable in its original state. In our case, this translates to switching to bus level one for all substations. Therefore, it is beneficial to return to the original state of the grid.

[8] Note that the observation of the *Tutor* were only a subset of all available observations, see Appendix A for more information.

[9] CurriculumAgent: https://github.com/FraunhoferIEE/CurriculumAgent (last access 20/03/2023).

[10] Note that while we only disconnect a subset of lines, the action itself might have an effect on other lines as well. Thus, we check the $\hat{\rho}^{(i,l)}$ for the whole grid.

Algorithm 1 The N-1 algorithm in pseudo code:

for all $i = 1, \dots, N$ **do**
  Select $a_i^{(topo)}$
  for all $l = 1, \dots, M$ **do**
    $a_l^{(line)} \leftarrow$ disconnect line l
    $a_{i,l} \leftarrow a_i^{(topo)} \wedge a_l^{(line)}$
    $\hat{\rho}_{j,t+1}^{(i,l)} \leftarrow \text{simulate}(a_{i,l})$
    $\hat{\rho}_{max,t+1}^{(i,l)} \leftarrow \max_{j=1,\dots,L}(\hat{\rho}_{j,t+1}^{(i,l)})$
  end for
  $\hat{\rho}_{max}^{(i,max)} = \max_{l=1,\dots,M}(\rho_{max,t+1}^{(i,l)})$
end for
$i^* \leftarrow \underset{i=1,\dots,N}{\text{argmin}}(\hat{\rho}_{max}^{(i,max)})$
return $a_{i^*}^{(topo)}$

The improvement is implemented as follows. When the maximum line capacity falls below the reversion threshold $\rho_{rev} = 0.8$, meaning no imminent danger is present, the greedy agent automatically searches through all substations and checks, whether their topology had been changed.[11] The agent then compares both the continuation of the current topology and a reversion to the original state in a simulation and chooses the better candidate. If multiple options are possible, the reversion with the lowest $\hat{\rho}_{max,t+1}$ is selected.

5.3. Code improvements

The last enhancements were code improvements to make the Binbinchen approach usable for the L2RPN-Baselines.[12] This included the requirement to make the agent compatible for all Grid2Op environments. Thus, hard coded lines and environment-specific solutions had to be reworked. In addition to the major changes, we added smaller features, such as filters of the observation array and the ability to add scalers from the Scikit-learn package for normalisation.[13] Furthermore, RLlib [30] was added to ensure a more flexible training with different RL algorithms. Supplementary to the RLlib enhancement, we also equipped both neural networks with hyperparameter tuning, which in case of the *Junior* model was the BOHB algorithm [31] and in case of the *Senior* the PBT algorithm [32]. Finally, we provided a test suite and ensured thorough documentation by adding docstrings, typehints, a readme and examples to our code to make the usability easier

6. Research design

While creating the evaluation of the experiments, we could observe that the performance within the test scenarios varied significantly and was strongly influenced by the selected seed of the environment. Therefore, we did not evaluate the agents on one specific seed but instead chose a more robust research design. We randomly generated thirty seeds, which in turn were fed into the evaluation procedure.[14] This setting ensured that the performance can be interpreted more reliably and we recommend this procedure for other researchers as well. With the proposed improvements of Section 5, we compare a total of six agents, which are described as follows:

[11] The ρ_{rev} is explicitly lower than the ρ_{tutor} because we want to make sure that the grid is stable and no agent action is required.

[12] L2RPN Baselines: https://l2rpn-baselines.readthedocs.io/en/latest/ (last access 20/03/2023).

[13] Scikit-learn package: https://scikit-learn.org/stable/ (last access 20/03/2023).

[14] The choice of a total of 30 seeds was a compromise between a sufficiently high degree of freedom and the computational cost of the evaluation. For the evaluation we use the internal method `ScoreL2RPN2020` of Grid2Op.

Table 1
Summary of the agents' results. All agents were run on the robustness track of the 2020 L2RPN test environment (24 scenarios) with thirty different seeds. The performance across the seeds is recorded below. We list the mean and the standard deviation in the first column and the median as well as the 25% and 75% quantile in the second column. Note that the *Do_Nothing* agent achieves a score of 0.00 per default.

Agent	Mean (Sd)	Median (Q25, Q75)
$Do\_Nothing$	00.00 (0.00)	00.00 (00.00, 00.00)
$Expert$	26.45 (4.52)	26.69 (24.03, 29.88)
$Tutor_{original}$	38.44 (4.19)	37.89 (35.75, 40.81)
$Tutor_{N-1}$	41.88 (4.11)	40.80 (38.97, 45.75)
$Tutor_{N-1,Topo}$	48.90 (4.67)	48.39 (44.67,53.40)
$Senior_{N-1,Topo}$	**49.12** (4.08)	**48.70** (45.53, 51.15)

Table 2
Test Results of the Welch's t-test [33] with the hypothesis $H_0 : \mu_i = \mu_j$ against the alternative hypothesis $H_1 : \mu_i \neq \mu_j$. For the normality assumption we tested with D'Agostino test [34] and could not reject the H_0 hypothesis, so non-normality could not be suspected.

H_0 Hypothesis	p-value
$H_0 : \mu_{T_o} = \mu_{T_{N-1}}$	0.0022
$H_0 : \mu_{T_o} = \mu_{T_{N-1Topo}}$	8.7e−13
$H_0 : \mu_{T_{N-1}} = \mu_{T_{N-1Topo}}$	6.9e−08
$H_0 : \mu_S = \mu_{T_{N-1Topo}}$	0.8454

1. To gain a baseline result, we provide the $Do\_Nothing$ as well as the $Expert$ of [25] introduced in Section 2.
2. Furthermore, we include $Tutor_{original}$ of the Binbinchen solution to our analysis.
3. Regarding the enhancements, we propose a $Tutor_{N-1}$ and a $Tutor_{N-1,Topo}$.
4. Lastly, we also provided a $Senior_{N-1,Topo}$, based on the $Tutor_{N-1,Topo}$. This $Senior_{N-1,Topo}$ is a hybrid between the RL model and heuristic methods (line reconnection, topology reversion).

Note that we only included the $Tutor_{original}$ of the Binbinchen solution for two reasons. First, we enhanced primarily the *Tutor* agent, thus comparing the tutors is most interesting. Second, on the Grid2Op version 1.7.1, the Binbinchen *Senior* did perform poorly on all thirty seeds.

For both the $Tutor_{N-1}$ and $Tutor_{N-1,Topo}$ improvements, as well as the $Senior_{N-1,Topo}$ it was necessary to compute the N-1 search, which was done by an N-1 *Teacher*. The resulting 300 actions were added to the base action set of 208 actions. In terms of the $Senior_{N-1,Topo}$, the agent was trained with all 508 actions (208 and 300 N-1) on the RLlib [30] framework. A hyperparameter training was included for both *Junior* and *Senior* models, as described in Section 5.3. The hyperparameter can be found in Appendix B. To prevent cherry picking, we run three experiments in parallel and evaluate the checkpoints in a validation environment, based on the training scenarios. We then select the best performing checkpoint across all three experiments.

7. Results

7.1. Experiment results

Using the experimental framework outlined above, the agents were evaluated on the thirty random seeds. The results can be found in Table 1, where we summarise the scores across seeds. A larger table and a Boxplot can be found in the Appendix C. In terms of our research question, we are primarily interested in the performance of the rule-based agents. Here, we could observe a clear improvement. While the $Tutor_{original}$ reached a mean score of 38.44, the $Tutor_{N-1}$ reached 41.88

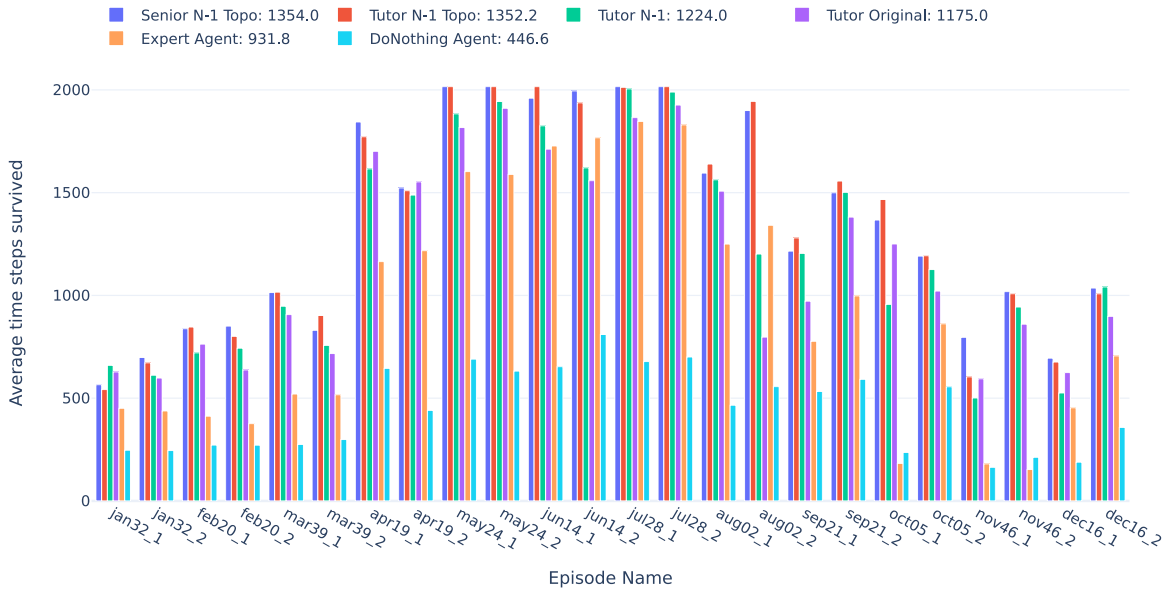
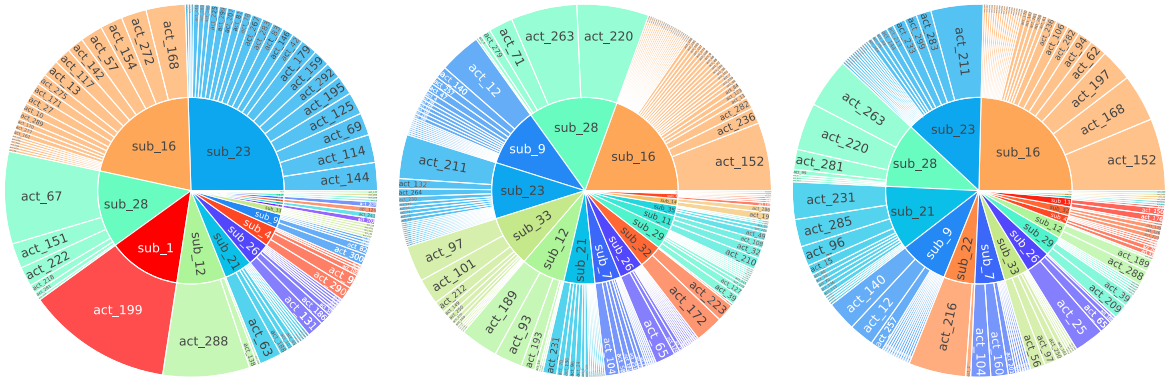


Fig. 3. Visualisation of the average survival time from the $Senior_{N-1,Topo}$ (blue), the $Tutor_{N-1,Topo}$ (red), the $Tutor_{N-1}$ (green), the $Tutor_{original}$ (purple), the $Expert$ (orange) and the $Do\_Nothing$ (turquoise). Each bar represents the average survival time of the agent in the respective scenario across all seeds. The overall average is reported in the legend. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)



(a) Actions of $Tutor_{original}$ **(b)** Actions of $Tutor_{N-1,Topo}$ **(c)** Actions of $Senior_{N-1,Topo}$

Fig. 4. Visualisation of the different topological actions of the agents $Tutor_{original}$, $Tutor_{N-1,Topo}$ and $Senior_{N-1,Topo}$. The actions are sorted according to the frequency of their substations. The most frequently used substation is at the top right, then all the other substations are ordered counterclockwise. The colour coding is the same for all three agents. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

and the $Tutor_{N-1,Topo}$ even 48.90, which is an increase by 27%. We tested with the Welch's t-test [33], whether the results of the different *Tutors* are from the same distribution and were able to reject the H_0 hypothesis for all Tutors, see Table 2. This shows that both the N-1 strategy and the reversion to the topology significantly improved the score of the agents. Further, the mean and median are relatively similar, thus we cannot directly detect a high influence of outliers. However, on can see in the quantiles that there is indeed a large variation in the performance depending on the seed.

Next to the *Tutors*, we also included the $Senior_{N-1,Topo}$ in our experiments to evaluate the impact of the RL algorithms. In this regard, we observe that the agent was slightly better than the $Tutor_{N-1,Topo}$ with a score of 49.12. However, the improvement was only fractional and not statistically significant, as shown in Table 2. Further note that the $Senior_{N-1,Topo}$ had a better performance in the median and the 25% quantile, but the $Tutor_{N-1,Topo}$ with 53.40 a better 75% quantile. This is incredibly interesting, given that researchers often postulate a clear superiority of RL approaches, which we could not replicate. Instead, it seems that the success is rather highly depending on the correct action sets. With respect to the baseline agents, the $Do\_Nothing$ and $Expert$ did not reach the score of the other agents, which is not surprising.

After evaluating the scores, we are also interested in the survival times of the different approaches. In Fig. 3 we show the average survival times across the 30 seeds, divided into the 24 test scenarios. Note that all scenarios ended after 2006 steps. If an agent was able to reach an average of 2006 steps for a scenario, this means that it was able to complete the scenario in every run. One can see that the performance across the scenarios differed between the agents, but there were also some similarities. The winter scenarios from November up until March were harder to solve by all participants, while the summer scenarios were easier to complete. In terms of survival, only a handful were completely survived by some of the agents. Both $Senior_{N-1,Topo}$ and $Tutor_{N-1,Topo}$ were able to complete four scenarios in all seeds, with $Senior_{N-1,Topo}$ completing the scenarios $(may24_1, may24_2, jul28_1, jul28_2)$ and $Tutor_{N-1,Topo}$ completing the scenarios $(may24_1, may24_2, jun14_1, jul28_2)$. All other agents failed to achieve this task. Nevertheless, $Tutor_{N-1}$ was almost able to complete $jul28_1$. This shows a clear performance increase.

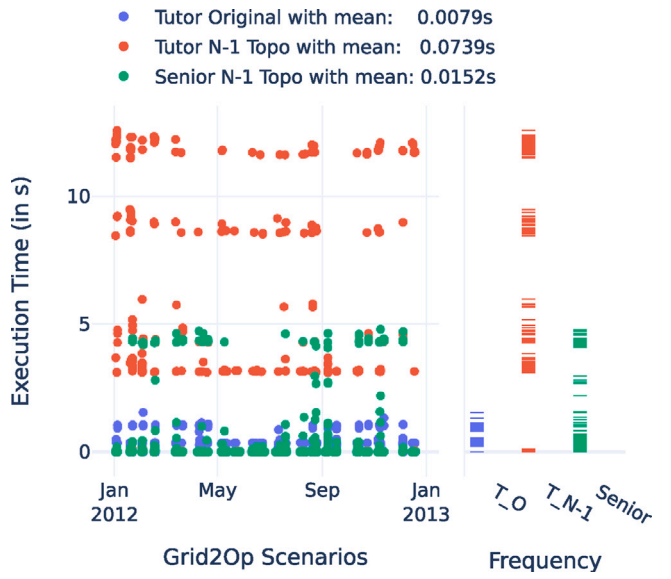


Fig. 5. Display of the computation time for each action. The left graph shows the computation time, where each point is one action of the agent. The vertical lines correspond to a specific scenario. On the right, we aggregate the computation time across all scenarios in a rug plot. Note that for comparison, we only include the computation times if all three agents survived until the given time step.

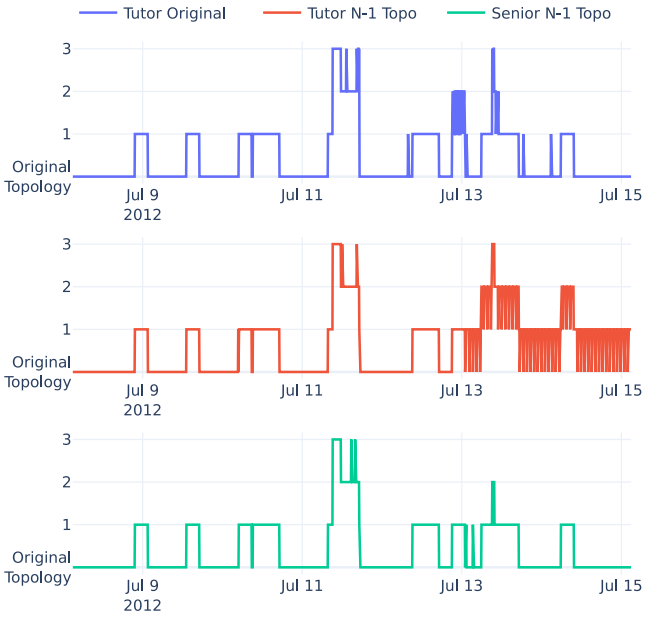


Fig. 6. Distance in Topology for the first July scenario. The y-axis describes the number of substations that differ in comparison to the original topology. Note that all agents completed the scenario.

7.2. Detailed behaviour analysis of the agents

After analysing the overall performance across the thirty seeds, we further provide a more detailed evaluation regarding the behaviour of the agents. Thus, we randomly select a run and looked at various metrics available in Grid2Op.[15] To begin with, we are interested in individual topological behaviour on a substation level. Hence, in Fig. 4 we show the topological actions sorted by their specific substation. First of all, we can see that the $Tutor_{original}$ executed actions from only five substations in more than 80% of the time. In contrast, the distributions between the substations of the $Tutor_{N-1,Topo}$ and the $Senior_{N-1,Topo}$ were more diversified with the 300 N-1 actions. Based on the grid of Fig. 2, we see that all three agents centred their actions around the three major nodes 16, 23 and 28, which were necessary to keep the grids stable. However, while the substation 23 took about one fourth of the overall actions from the $Tutor_{original}$, the N-1 agents rather preferred topological changes on the substation 16. It is also interesting to note that the $Tutor_{original}$ only had one action at substation one, which it used frequently. The N-1 agents, on the other hand, did not rely on this substation at all.

When comparing the behaviour between the rule based $Tutor_{N-1,Topo}$ and the $Senior_{N-1,Topo}$, we can see that there are several similarities but also slight changes in the behaviour. One can observe that the rule-based agent utilised the substations 33 and 9 to divert power from substation 16 to the south-east and north-west. In contrast, we see that these substations rank lower for the RL agent and instead the $Senior_{N-1,Topo}$ diverted the power to the substations 23 and 21, i.e., corresponding to a north-eastern route. As a result, the $Senior_{N-1,Topo}$ had to rely on substation 22 more often. This shows a different learned strategy by the RL approach, with the same potential outcome.

[15] The randomly selected seed had a similar performance to the average score of Section 7.1. Thus, we expect the behaviour to be comparable. Note that the data extraction is based on adjusted methods of the package Grid2Bench https://github.com/IRT-SystemX/Grid2Bench (last access 20/03/2023).

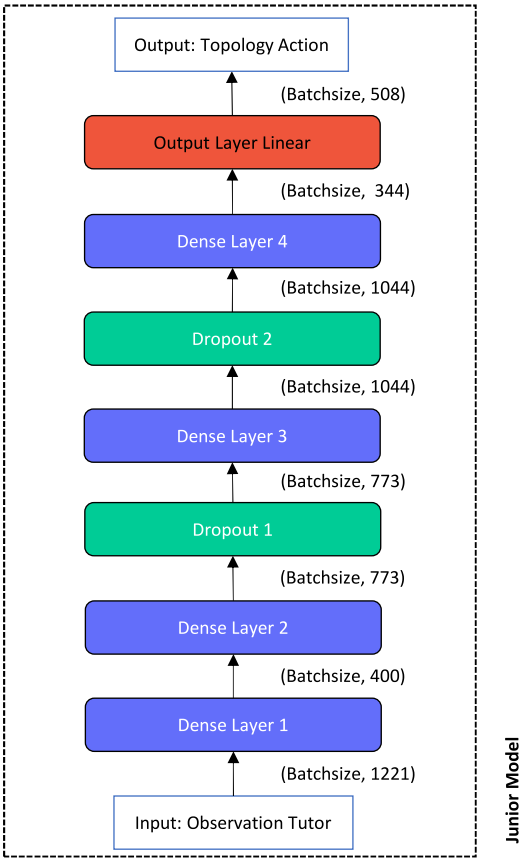


Fig. B.7. Visualisation of the Junior Model after hyperparameter training, which included the number of neurons. In the original paper, all four hidden models had a total of 1000 layers. The hyperparameter after optimisation are listed in the Table B.4 below. The observation of the Tutor model and the output corresponds to the number of actions, i.e., 208 actions from Binbinchen and 300 N-1 actions.

Table A.3
In order to not inflate the input of the deep learning models, the observations of the Tutor were filtered to include only the relevant data. Consequently, the experience of the *Tutor* as well as the input of the *Junior* and *Senior* all had the reduced observation, which in case of the L2RPN robustness track corresponds to a size of (1221,). This process was copied from Binbinchen [9]. The following information was included in the observation of the *Tutor*, *Junior* and *Senior*. The description is from the Grid2Op documentation and can be found under https://grid2op.readthedocs.io/en/latest/observation.html.

Category	Variable	Description
Time Values	$month$	Month of the Scenario
	day	Day in time step t
	$hour\_of\_day$	Hour of time step t
	$minute\_of\_hour$	Minute of time stpe t
	day_of_week	Weekday of time step t
Generation and Load	gen_p, gen_q	Active and reactive production value of each generator
	gen_v	Voltage magnitude of each generator at connected bus
	$load\_p, load\_q$	Active and reactive load value of each consumption
	gen_v	Voltage magnitude of each consumption at connected bus
Lines	p_or, q_or	Active and reactive power flow at the origin end of each power line
	v_or	Voltage magnitude at the origin end of each power line
	a_or	Current flow at the origin end of each power line
	p_ex, q_ex	Active and reactive power flow at the extremity end of each power line
	v_ex	Voltage magnitude at the extremity end of each power line
	a_ex	Current flow at the extremity end of each power line
	rho or ρ	capacity of each power line
	$line\_status$	Whether the line is connected or disconnected
	$timestep\_overflow$	Number of time steps since overflow
Bus Information	$topo\_vect$	Vector of the topology configurations
Cooldowns	$time\_before\_cooldown\_line$	Time before an action could be done on the line
	$time\_before\_cooldown\_sub$	Time before an action could be done on the substation
Maintenance	$time\_next\_maintenance$	Time when the next maintenance of line is planned
	$duration\_next\_maintenance$	Duration of planed maintenance

Next to the different routing, we want to highlight different computational times of the agents. In Fig. 5, we plotted for each step the required computation time of the agent, which we summarised in the rug plot on the right side. The first thing that becomes clear is that the $Tutor_{N-1,Topo}$ increased the computation time quite a bit compared to the $Tutor_{original}$. This can be explained by the fact that the $Tutor_{original}$ had the 208 actions divided into two sets of size 62 and 146 and first only evaluated the 62 actions. Only if no adequate candidate was found, it continued with the 146 action set. Further, no additional heuristics were added to the tutor. In contrast, the $Tutor_{N-1,Topo}$ started with the N-1 action set consisting of 300 actions, then continued with 62 and the 146 actions. Consequently, the longer survival of $Tutor_{N-1,Topo}$ came at a cost in terms of computation time.

Compared to the $Tutor_{N-1,Topo}$, the $Senior_{N-1,Topo}$ was faster than the rule-based approach. This is due to the fact that the $Senior_{N-1,Topo}$ did not have a clear division of its action sets, as it provided a policy for all 508 (208+300) actions. This made it easier to find a suitable candidate and the search was accelerated without any loss of accuracy. Note that the actions with a computational time of almost zero correspond to the do-nothing actions, which all three agents selected when the grid was stable.

As a last result, we also want to look at the distance from the original topology, which we visualised exemplary for the July week $jul28\_1$ in Fig. 6. The distance was measured in the number of changed substations in comparison to the original topology configuration. Here, there are two interesting things to note. First, we can see that all agents had a relatively similar behaviour until the 12th of July, moving up to three steps away from the original topology and afterwards returning to it. However, we see a clear difference between the *Tutors* and the $Senior_{N-1,Topo}$ at the start of July 13th. While the $Senior_{N-1,Topo}$ had only a maximum distance of two topology changes, the *Tutors* had a distance of three, thus showing that the $Senior_{N-1,Topo}$ found a better action in that moment. A second thing to note is that the $Tutor_{N-1,Topo}$ showed very unstable behaviour after 13 July, which is not ideal for a real substation. A possible explanation could be that the $\rho_{max,t}$ of the grid was fluctuating around the threshold ρ_{tutor} of the $Tutor_{N-1,Topo}$, thus triggering the need to act. Both results are very interesting, because they show that the *Senior* was able to achieve

Table B.4
Hyperparameter of the Junior Model after running the Hyperparameter search with the BOHB [31].

Specification	Parameter	Value
Model architecture	Hidden Layer 1	400 Neurons
	Hidden Layer 2	773 Neurons
	Hidden Layer 3	1044 Neurons
	Hidden Layer 4	344 Neurons
	Output Layer Linear	508 Neurons
	Activation	Relu Activation
	Batchsize	768
	Dropout 1	0.157474
	Dropout 2	0.408924
	Initialiser	Orthogonal
	Learning Rate	0.000128
	Epochs	1000
	Early stopping	50 steps

Table B.5
Hyperparameter of the Senior model. The model is trained with the PPO algorithm in combination with PBT [32]. The underlying structure of the RL model is the same as the Junior model in Fig. B.7. As training framework we use RLlib [30].

Specification	Parameter	Value
PPO Specifications	Learning Rate	1e−05
	Clip-Parameter	0.07267018
	Entropy Coefficient	0.00096065
	Gamma	0.99236394
	VF loss coefficient	0.84545350
	Number of SGD steps	4
	SGD Minibatch Size	128(default)
	KL Coefficient	0.2 (default)
Model architecture	Hidden Layer 1	400 Neurons
	Hidden Layer 2	773 Neurons
	Hidden Layer 3	1044 Neurons
	Hidden Layer 4	344 Neurons
	Activation	Relu Activation
	Initialiser	Orthogonal

Table C.6
Large summary table, similar to Table 1 of the agents' results. All agents were run on the robustness track of the 2020 L2RPN test environment(24 scenarios) with thirty different seeds. The performance across the seeds is recorded below. We list the mean, the standard deviation, the median, the 25% and 75% quantile and the minimum and maximum values across the seeds. Note that the *Do_Nothing* agent achieves a score of 0.00 per default.

	$Do\_Nothing$	$Expert$	$Tutor_{original}$	$Tutor_{N-1}$	$Tutor_{N-1,Topo}$	$Senior_{N-1,Topo}$
mean	0.0	26.45	38.44	41.88	48.90	49.12
std	0.0	4.52	4.19	4.11	4.67	4.08
Min	0.0	16.95	30.46	34.54	41.80	41.82 25
% Quantile	0.0	24.03	35.75	38.97	44.67	45.53
Median	0.0	26.69	37.89	40.80	48.39	48.70
75% Quantile	0.0	29.88	40.81	45.75	53.40	51.15
Max	0.0	35.96	47.42	51.61	58.23	58.69

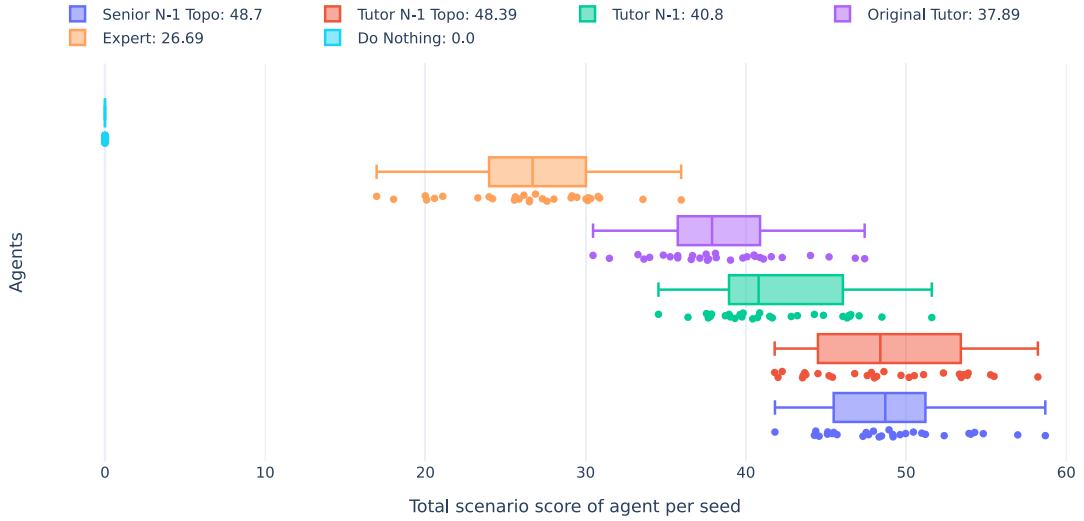


Fig. C.8. Boxplot of the agent results across all 30 seeds. Each point corresponds to the total score of all scenarios for one seed. In the legend we report the median across all seeds. Note that the *Do_Nothing* agent scores per default 0.00.

a similar effect, however used a different topology action. Here, less distance from the original grid was required and the overall state was more stable.

7.3. Discussion and future outlook

As outlined in the previous Section 7, we were able to significantly improve the original greedy search *Tutor* by providing both an active revision of the topology as well as an additional N-1 strategy. In order to obtain more advanced rule-based approaches, we showed that classical power-grid strategies can be beneficial as improvements.

When comparing the more sophisticated rule-based agent with the RL approach, we were not able to demonstrate a clear superiority. Even though the RL agent was slightly better, one could argue that this may not have enough statistical significance. This is particularly interesting when considering that heuristic methods were also included in the $Senior_{N-1,Topo}$. This shows that strategies such as reconnecting lines and simulating prior to the execution of an action have a high proportion of the overall performance score. Nevertheless, we still consider RL to be a necessity for future network operations for the following three reasons:

First, the RL approach clearly showed an advantage in the computational speed, as was shown in Fig. 5. This is quite interesting considering that the agent had no direct goal to use N-1 actions like the $Tutor_{N-1,Topo}$. However, the agent was still able to learn a similar behaviour, demonstrating the ability to imitate desired behaviour. This feature may be crucial, especially for larger grids. Considering that an increase in the number of available actions could push rule-based approaches to their limits, RL can still compute solutions in an acceptable time. Second, in order to replicate the Binbinchen approach we only used the PPO algorithm in this work. We therefore did not focus on specific RL improvements. As other researchers showed, e.g., in [23], there are advanced RL techniques available that can enhance the RL models. Thus, it could be possible to further increase the score through different RL agents. Third, it can be advantageous to consider multiple approaches simultaneously for ensemble methods. As analysed in Section 7.2, the behaviour of the $Senior_{N-1,Topo}$ differed from that of the $Tutor_{N-1,Topo}$ agent, thus it could provide alternative strategies for grid operators.

As a consequence, although the main development of this article was a more sophisticated rule-based agent, we encourage other researchers to further improve RL approaches. One possibility might be a split of the $Senior_{N-1,Topo}$ into two independent agents, in order to gain more specialised agents, e.g., one N-1 agent and one agent for extreme cases ($\rho_{max} > 1.0$).

8. Conclusion

In this paper, we analysed the structure of the Binbinchen agent from the 2020 L2RPN robustness Challenge and proposed additional improvements to the rule-based greedy agent. The novelty of our improvements was on the one hand the N-1 strategy and the reversion back to the base topology of the agent. In order to evaluate the proposed improvements, we tested the agents on the L2RPN test environment with 30 different seeds and additionally evaluated one random seed in particular. Our experiments show that the improvements increased the performance by 27% and that the N-1 rule-based agent was able to achieve a similar result to the RL agent. In our detailed analysis, we also showed that by considering the N-1 actions, the overall use of actions by both the rule-based N-1 agent and the RL agent became more diverse compared to the original agent, leading

to an increase in stability of the grid. We discussed these results, highlighting the comparison between the rule-based approach and the RL agent.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code can be found under https://github.com/FraunhoferIEE/CurriculumAgent and will be uploaded after acceptance of the paper.

Acknowledgements

This work was supported by the Competence Centre for Cognitive Energy Systems of the Fraunhofer IEE and the research group Reinforcement Learning for cognitive energy systems (RL4CES) from the Intelligent Embedded Systems of the University Kassel.

Appendix A. Filter of the Tutor model

See Table A.3.

Appendix B. Structure deep learning models

B.1. Visualisation Junior model

See Fig. B.7.

B.2. Hyperparameter Junior model

See Table B.4.

B.3. Hyperparameter Senior agent

See Table B.5.

Appendix C. Visualisation results

See Fig. C.8 and Table C.6.

References

[1] Marot A, Kelly A, Naglic M, Barbesant V, Cremer J, Stefanov A, et al. Perspectives on future power system control centers for energy transition. J Mod Power Syst Clean Energy 2022;10(2):328–44.
[2] Bacher R, Glavitsch H. Network topology optimization with security constraints. IEEE Trans Power Syst 1986;1(4):103–11.
[3] Viebahn J, Naglic M, Marot A, Donnot B, Tindemans SH. Potential and Challenges of AI-Powered Decision Support for Short-Term System Operations. CIGRE; 2022.
[4] Capitanescu F. Critical review of recent advances and further developments needed in AC optimal power flow. Electr Power Syst Res 2016;136:57–68.
[5] Marot A, Guyon I, Donnot B, Dulac-Arnold G, Panciatici P, Awad M, et al. L2rpn: Learning to run a power network in a sustainable world neurips2020 challenge design. 2020.
[6] Marot A, Donnot B, Romero C, Donon B, Lerousseau M, Veyrin-Forrer L, et al. Learning to run a power network challenge for training topology controllers. Electr Power Syst Res 2020;189:106635.
[7] Donnot B. Grid2op- A testbed platform to model sequential decision making in power systems. 2020, https://GitHub.com/rte-france/grid2op. (Accessed 22 January 2023) on Github.
[8] Marot A, Donnot B, Dulac-Arnold G, Kelly A, O'Sullivan A, Viebahn J, et al. Learning to run a power network challenge: a retrospective analysis. In: NeurIPS 2020 competition and demonstration track. PMLR; 2021, p. 112–32.
[9] EI Innovation Lab, Huawei Cloud, Huawei Technologies. NeurIPS competition 2020: Learning to run a power network (L2RPN) - robustness track. 2020, https://github.com/AsprinChina/L2RPN_NIPS_2020_a_PPO_Solution. (Accessed 22 January 2023) on Github.
[10] Ernst D, Glavic M, Wehenkel L. Power systems stability control: reinforcement learning framework. IEEE Trans Power Syst 2004;19(1):427–35.
[11] Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature 2015;518(7540):529–33.
[12] Mnih V, Badia AP, Mirza M, Graves A, Lillicrap T, Harley T, et al. Asynchronous methods for deep reinforcement learning. In: International conference on machine learning. PMLR; 2016, p. 1928–37.
[13] Lee J, Hwangbo J, Wellhausen L, Koltun V, Hutter M. Learning quadrupedal locomotion over challenging terrain. Science Robotics 2020;5(47):eabc5986.
[14] Schrittwieser J, Antonoglou I, Hubert T, Simonyan K, Sifre L, Schmitt S, et al. Mastering atari, go, chess and shogi by planning with a learned model. Nature 2020;588(7839):604–9.
[15] Berner C, Brockman G, Chan B, Cheung V, Debiak P, Dennison C, et al. Dota 2 with large scale deep reinforcement learning. 2019, arXiv preprint arXiv:1912.06680.
[16] Kelly A, O'Sullivan A, de Mars P, Marot A. Reinforcement learning for electricity network operation. 2020, arXiv preprint arXiv:2003.07339.
[17] Subramanian M, Viebahn J, Tindemans SH, Donnot B, Marot A. Exploring grid topology reconfiguration using a simple deep reinforcement learning approach. In: 2021 IEEE madrid PowerTech. 2021, p. 1–6. http://dx.doi.org/10.1109/PowerTech46648.2021.9494879.
[18] Lan T, Duan J, Zhang B, Shi D, Wang Z, Diao R, et al. AI-based autonomous line flow control via topology adjustment for maximizing time-series ATCs. In: 2020 IEEE Power & Energy Society general meeting. IEEE; 2020, p. 1–5.
[19] Yoon D, Hong S, Lee B-J, Kim K-E. Winning the l2rpn challenge: Power grid management via semi-markov afterstate actor-critic. 2021.
[20] Zhou B, Zeng H, Liu Y, Li K, Wang F, Tian H. Action set based policy optimization for safe power grid management. In: Machine learning and knowledge discovery in databases. Applied data science track: European conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, proceedings, part V 21. Springer; 2021, p. 168–81.
[21] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017, arXiv preprint arXiv:1707.06347.
[22] Chauhan A, Baranwal M, Basumatary A. PowRL: A reinforcement learning framework for robust management of power networks. 2022, arXiv preprint arXiv:2212.02397.
[23] Dorfer M, Fuxjäger AR, Kozak K, Blies PM, Wasserer M. Power grid congestion management via topology optimization with AlphaZero. 2022, arXiv preprint arXiv:2211.05612.
[24] Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. 2017, arXiv preprint arXiv:1712.01815.
[25] Marot A, Donnot B, Tazi S, Panciatici P. Expert system for topological remedial action discovery in smart grids. IET; 2018, HAL.
[26] Brockman G, Cheung V, Pettersson L, Schneider J, Schulman J, Tang J, et al. Openai gym. 2016, arXiv preprint arXiv:1606.01540.
[27] Omnes L, Marot A, Donnot B. Adversarial training for a continuous robustness control problem in power systems. In: 2021 IEEE madrid PowerTech. IEEE; 2021, p. 1–6.
[28] Jixiang L, Zhihong Y, Chunlei X, Hongsheng X, Kang X, Tianyu C, et al. A winning approach of NeurIPS 2020 L2RPN comp. 2020, https://github.com/lujasone/NeurIPS_2020_L2RPN_Comp_An_Approach. (Accessed 19 March 2023) on Github.
[29] Marot A, Donnot B, Chaouache K, Kelly A, Huang Q, Hossain R-R, et al. Learning to run a power network with trust. Electr Power Syst Res 2022;212:108487.
[30] Liang E, Liaw R, Nishihara R, Moritz P, Fox R, Goldberg K, et al. RLlib: Abstractions for distributed reinforcement learning. In: Dy J, Krause A, editors. Proceedings of the 35th international conference on machine learning. Proceedings of machine learning research, vol. 80, PMLR; 2018, p. 3053–62, URL https://proceedings.mlr.press/v80/liang18b.html.
[31] Falkner S, Klein A, Hutter F. BOHB: Robust and efficient hyperparameter optimization at scale. In: International conference on machine learning. PMLR; 2018, p. 1437–46.
[32] Jaderberg M, Dalibard V, Osindero S, Czarnecki WM, Donahue J, Razavi A, et al. Population based training of neural networks. 2017, arXiv preprint arXiv:1711.09846.
[33] Welch BL. The generalization of 'STUDENT'S'problem when several different population varlances are involved. Biometrika 1947;34(1–2):28–35.
[34] D'agostino R, Pearson ES. Tests for departure from normality. Biometrika 1973;60(3):613–22.